Complete End-to-End Technical Process

The Complete End-to-End Technological Process
Attlee BSR Quality Checker: From PDF Upload to Amended Documents
Overview: The Complete Journey
Stage 1: PDF Upload & Text Extraction
Stage 2: Document Chunking & Classification
Stage 3: AI Field Extraction (Claude Sonnet 4.5)
Stage 4: Deterministic Rules Evaluation (55 Checks)
Stage 5: AI Nuanced Analysis (LLM Assessments)
Stage 6: Combined Results & Readiness Scoring
Stage 7: AI Document Generation (Amended Docs + PDFs)
Complete End-to-End Summary
Appendix: Key Code Files

The Complete End-to-End Technological Process

Attlee BSR Quality Checker: From PDF Upload to Amended Documents

---

Overview: The Complete Journey

This document explains the **complete technological pipeline** from the moment a user uploads PDF documents to when they download submission-ready amended documents.

The 7-Stage Pipeline:

```
Stage 1: PDF Upload & Text Extraction
         ↓
Stage 2: Document Chunking & Classification
         ↓
Stage 3: AI Field Extraction (Claude Sonnet 4.5)
         ↓
Stage 4: Deterministic Rules Evaluation (55 checks)
         ↓
Stage 5: AI Nuanced Analysis (LLM assessments)
         ↓
Stage 6: Combined Results & Readiness Scoring
         ↓
Stage 7: AI Document Generation (Amended docs + PDFs)
```

Total Time: 3-5 minutes for a typical 12-document submission pack

---

Stage 1: PDF Upload & Text Extraction

What Happens

User uploads PDF documents (typically 10-20 files: Fire Strategy, Structural Reports, Drawings, etc.)

The Technology

File: `packages/backend/src/services/ingestion.ts`

```
import pdfParse from 'pdf-parse';

async function ingestDocument(filepath: string) {
  // Read the PDF file from disk
  const dataBuffer = fs.readFileSync(filepath);

  // Extract text using pdf-parse library
  const pdfData = await pdfParse(dataBuffer);

  return {
    text: pdfData.text,           // Full extracted text
    pageCount: pdfData.numpages,  // Number of pages
    isScanned: detectIfScanned(pdfData.text) // OCR quality check
  };
}
```

What Gets Extracted

- **Full text content** from every page
- **Page count** (for citation references later)
- **Scanned document detection** (if text extraction fails, flags as scanned/image-based)

Data Storage

```
-- Documents table stores metadata
INSERT INTO documents (
  id,
  filename,
  filepath,
  pageCount,
  isScanned,
  packVersionId
) VALUES (...);
```

Real Example

Input: `Fire_Strategy_Report_v2.pdf` (84 pages, 2.3 MB)

Output:

```
{
  "filename": "Fire_Strategy_Report_v2.pdf",
  "pageCount": 84,
  "isScanned": false,
  "textLength": 156743  // characters extracted
}
```

Stage 2: Document Chunking & Classification

What Happens

The extracted text is: 1. **Classified** by document type (Fire Strategy, Structural, MEP, etc.) 2. **Split into chunks** for efficient processing 3. **Stored in database** with metadata

The Technology

File: `packages/backend/src/services/ingestion.ts`

2.1 Document Classification

```
function classifyDocType(filename: string, text: string): string | null {
  const lowerFilename = filename.toLowerCase();
  const lowerText = text.toLowerCase().slice(0, 5000); // First 5000 chars

  const typePatterns = [
    {
      type: 'fire_strategy',
      patterns: ['fire strategy', 'fire safety strategy', 'fire engineering']
    },
    {
      type: 'structural',
      patterns: ['structural', 'structure', 'load', 'foundation']
    },
    {
      type: 'mep',
      patterns: ['mechanical', 'electrical', 'plumbing', 'mep', 'hvac']
    },
    // ... 10 total document types
  ];

  // Check filename and first 5000 characters for pattern matches
  for (const { type, patterns } of typePatterns) {
    for (const pattern of patterns) {
      if (lowerFilename.includes(pattern) || lowerText.includes(pattern)) {
        return type;
      }
    }
  }

  return null; // Unclassified
}
```

Why this matters: Deterministic rules later need to know “Is there a Fire Strategy document?” This classification enables that check.

2.2 Text Chunking

```
const CHUNK_SIZE = 1000;        // characters per chunk
const CHUNK_OVERLAP = 200;      // overlap between chunks

function chunkText(text: string, pageRef: number): ChunkData[] {
  const chunks: ChunkData[] = [];
  let index = 0;
  let chunkIndex = 0;

  while (index < text.length) {
    const end = Math.min(index + CHUNK_SIZE, text.length);
    const chunkText = text.slice(index, end).trim();

    chunks.push({
      text: chunkText,
      pageRef: pageRef,
      chunkIndex: chunkIndex
    });

    // Move forward by CHUNK_SIZE - CHUNK_OVERLAP
    index += (CHUNK_SIZE - CHUNK_OVERLAP);
    chunkIndex++;
  }

  return chunks;
}
```

Why chunking? - AI models have context limits (200K tokens for Claude) - Chunking allows processing long documents in parts - Overlap ensures context isn't lost at boundaries

Data Storage

```sql
-- Chunks table stores text segments
INSERT INTO chunks (
  id,
  documentId,
  text,
  chunkIndex,
  pageRef
) VALUES (...);

-- Example: 84-page Fire Strategy becomes ~150 chunks
```

Real Example

Input: 156,743 characters of text from Fire Strategy Report

Output:

```json
{
  "docType": "fire_strategy",
  "chunks": [
    {
      "chunkIndex": 0,
      "text": "Gateway 2 Fire Safety Strategy...",
      "pageRef": 1
    },
    {
      "chunkIndex": 1,
      "text": "...Strategy for Riverside Tower. The building...",
      "pageRef": 1
    },
    // ... 149 more chunks
  ]
}
```

Stage 3: AI Field Extraction (Claude Sonnet 4.5)

What Happens

Claude AI reads each document and extracts **key building information** like: - Building height - Number of storeys - Evacuation strategy - Sprinkler system presence - External wall materials - Smoke control systems - Basement levels - Stair core count - etc. (9 key fields tracked)

The Technology

File: `packages/backend/src/services/analysis.ts`

```
async function extractFieldsFromDocument(documentId: string) {
  // 1. Load document chunks from database
  const document = await prisma.document.findUnique({
    where: { id: documentId },
    include: { chunks: true }
  });

  // 2. Reconstruct full text (limit to 50K chars for API efficiency)
  const content = document.chunks
    .sort((a, b) => a.chunkIndex - b.chunkIndex)
    .map(c => c.text)
    .join('\n')
    .slice(0, 50000);

  // 3. Call Claude API with extraction prompt
  const response = await extractJSON<FieldExtractionResponse>(
    FIELD_EXTRACTION_SYSTEM_PROMPT,
    FIELD_EXTRACTION_USER_PROMPT(content, document.filename)
  );

  return response;
}
```

The AI Prompt

File: `packages/backend/src/prompts/extractFields.ts`

System Prompt:

```
You are an expert document analyst specializing in Building Safety Regulator (BSR)
Gateway 2 submission packs for high-rise residential buildings in England.

Your task is to extract key building information from the provided document content.
Focus on accuracy and providing evidence for each extracted field.

For each field, you must provide:
- fieldValue: The extracted value (or null if not found)
- confidence: "high", "medium", or "low"
- evidenceQuote: A direct quote from the document supporting the value
- pageRef: The page number where the evidence was found (if available)

Be conservative with confidence ratings:
- high: Clear, unambiguous statement in the document
- medium: Information is present but may require interpretation
- low: Information is implied or partially present
```

User Prompt:

```
Analyze the following document and extract the key building information fields.

Document: Fire_Strategy_Report.pdf
Content:
[First 50,000 characters of extracted text...]

Extract the following fields and return as JSON:
{
  "fields": [
    {
      "fieldName": "building_height",
      "fieldValue": string | null,
      "confidence": "high" | "medium" | "low",
      "evidenceQuote": string | null,
      "pageRef": number | null
    },
    {
      "fieldName": "number_of_storeys",
      ...
    },
    // ... 9 total fields
  ]
}

Only return valid JSON. Extract only what is explicitly stated or clearly implied.
```

AI Model Used

Model: `claude-sonnet-4-20250514` (Claude Sonnet 4.5) - **Context window:** 200K tokens - **Output:** Up to 4096 tokens - **Cost:** ~$3 per document extraction (enterprise pricing)

What AI Does

✅ **AI DOES:** - Read document text quickly - Identify specific information (height, storeys, etc.) - Quote evidence directly from source - Assign confidence scores

❌ **AI DOES NOT:** - Make pass/fail compliance decisions - Determine if requirements are met - Generate final judgments

Real Example

Input: Fire Strategy Report text

AI Output:

```
{
  "fields": [
    {
      "fieldName": "building_height",
      "fieldValue": "24.5m",
      "confidence": "high",
      "evidenceQuote": "The building has a total height of 24.5 metres above ground
        level as measured from the lowest ground level at the building perimeter",
      "pageRef": 12
    },
    {
      "fieldName": "evacuation_strategy",
      "fieldValue": "Stay Put",
      "confidence": "high",
      "evidenceQuote": "The building will adopt a stay put evacuation strategy in
        accordance with BS 9991:2015 for purpose-designed blocks of flats",
      "pageRef": 18
    },
    {
      "fieldName": "sprinkler_system",
      "fieldValue": "Yes - BS 9251:2014",
      "confidence": "high",
      "evidenceQuote": "An automatic sprinkler system will be installed throughout
        the building in accordance with BS 9251:2014",
      "pageRef": 24
    },
    {
      "fieldName": "smoke_control",
      "fieldValue": "Mechanical AOV system",
      "confidence": "medium",
      "evidenceQuote": "Smoke control within common corridors will be achieved
        through a mechanical AOV system with automatic activation",
      "pageRef": 31
    }
  ]
}
```

Data Storage

```sql
-- Store extracted fields in database
INSERT INTO extractedFields (
  packVersionId,
  fieldName,
  fieldValue,
  confidence,
  evidenceDocumentId,
  evidencePageRef,
  evidenceQuote
) VALUES (...);
```

Processing Time

- **Per document:** 5-15 seconds
- **Full pack (12 documents):** 60-180 seconds (run in parallel)

Stage 4: Deterministic Rules Evaluation (55 Checks)

What Happens

55 proprietary deterministic rules evaluate the extracted evidence and return **binary PASS/FAIL** decisions for regulatory requirements.

The Technology

File: `packages/backend/src/services/deterministic-rules.ts` (3,627 lines)

```
export function runDeterministicChecks(docs: DocumentEvidence[]):
        DeterministicAssessment[] {
  const results: DeterministicAssessment[] = [];

  // Run all 55 rules
  results.push(checkFireStrategyPresence(docs));
  results.push(checkHeightConsistency(docs));
  results.push(checkEvacuationStrategyDefined(docs));
  results.push(checkSprinklerRequirement(docs));
  results.push(checkSmokeControlSpecified(docs));
  // ... 50 more rules

  return results;
}
```

Rule Anatomy

Every rule follows this structure:

1. **Document Presence Check** - Is the required document there?
2. **Content Verification** - Does it contain required elements?
3. **Quality Assessment** - Is the content complete and specific?

Real Rule Example: Sprinkler System Check

```
function checkSprinklerRequirement(docs: DocumentEvidence[]):
        DeterministicAssessment {
  const MATRIX_ID = 'GW2-FS-008';
  const RULE_NAME = 'Sprinkler System - High Rise Buildings';

  // Step 1: Find Fire Strategy document
  const fireStrategy = findDocument(docs, ['fire strategy', 'fire safety']);

  if (!fireStrategy) {
    return {
      matrixId: MATRIX_ID,
      ruleName: RULE_NAME,
      category: 'Fire Safety',
      severity: 'high',
      result: {
        passed: false,
        confidence: 'definitive',
        evidence: {
          found: false,
          document: null,
          quote: null,
          matchType: 'absence'
        },
        reasoning: 'Fire Strategy document not found in submission',
        failureMode: 'Missing required document'
      },
      requiresLLMReview: false,
      regulatoryRef: {
        source: 'Building Regulations Approved Document B',
        section: 'Section 8: Fire Suppression',
        requirement: 'Buildings over 18m must have automatic sprinkler systems'
      }
    };
  }

  // Step 2: Extract building height
  const heights = extractHeights(fireStrategy.extractedText);

  if (heights.length === 0) {
    return {
      // ... result indicating height not found
      requiresLLMReview: true  // Flag for human review
    };
  }
```

```
const maxHeight = Math.max(...heights);

// Step 3: Check threshold
if (maxHeight <= 18) {
  return {
    matrixId: MATRIX_ID,
    ruleName: RULE_NAME,
    result: {
      passed: true,
      confidence: 'definitive',
      reasoning: `Building is ${maxHeight}m - sprinklers not mandatory for
      buildings ≤18m`
    }
  };
}

// Step 4: Building is over 18m - search for sprinkler mentions
const sprinklerKeywords = [
  'sprinkler',
  'automatic water suppression',
  'water mist',
  'ESFR',
  'BS 9251',
  'BS 12845'
];

let foundKeyword = null;
let quote = null;

for (const keyword of sprinklerKeywords) {
  if (containsKeyword(fireStrategy.extractedText, keyword)) {
    foundKeyword = keyword;
    quote = extractQuote(fireStrategy.extractedText, keyword, 150);
    break;
  }
}

if (foundKeyword) {
  return {
    matrixId: MATRIX_ID,
    ruleName: RULE_NAME,
    category: 'Fire Safety',
    severity: 'high',
    result: {
      passed: true,
      confidence: 'definitive',
      evidence: {
        found: true,
```

```
          document: fireStrategy.filename,
          quote: quote,
          matchType: 'keyword'
        },
        reasoning: `Building is ${maxHeight}m tall and sprinkler system is
        documented`,
        failureMode: null
      },
      requiresLLMReview: false
    };
  } else {
    return {
      matrixId: MATRIX_ID,
      ruleName: RULE_NAME,
      category: 'Fire Safety',
      severity: 'high',
      result: {
        passed: false,
        confidence: 'high',
        evidence: {
          found: false,
          document: fireStrategy.filename,
          quote: null,
          matchType: 'absence'
        },
        reasoning: `Building is ${maxHeight}m tall but no sprinkler system
        documentation found`,
        failureMode: 'High-rise building missing required fire suppression system'
      },
      requiresLLMReview: false
    };
  }
}
```

Utility Functions

```
// Extract all height values from text
function extractHeights(text: string): number[] {
  const regex = /(\d+(?:\.\d+)?)\s*(?:m|metres?|meters?)\s*
        (?:high|tall|height|above)/gi;
  const matches = text.match(regex) || [];
  return matches
    .map(m => parseFloat(m.match(/\d+(?:\.\d+)?/)?.[0] || '0'))
    .filter(h => h > 0);
}

// Check if text contains keyword (normalized)
function containsKeyword(text: string, keyword: string): boolean {
  const normText = text.toLowerCase().replace(/\s+/g, ' ').trim();
  const normKeyword = keyword.toLowerCase();
  return normText.includes(normKeyword);
}

// Extract quote with context
function extractQuote(text: string, keyword: string, contextChars: number): string {
  const idx = text.toLowerCase().indexOf(keyword.toLowerCase());
  if (idx === -1) return null;

  const start = Math.max(0, idx - contextChars);
  const end = Math.min(text.length, idx + keyword.length + contextChars);
  return '...' + text.slice(start, end).trim() + '...';
}
```

The 55 Rules

Breakdown by category:

Category	Rule Count	Example Rules
Document Presence	10	Fire Strategy present, Structural Report present, MEP specs present
Content Completeness	20	Height specified, Evacuation strategy defined, Sprinklers documented
Consistency Checks	15	Heights match across docs, Storey counts align, Fire ratings consistent
Cross-References	10	Fire Strategy references drawings, Structural cites calculations

Confidence Levels

Confidence	Meaning	When Assigned
Definitive	100% certain	Required keyword found explicitly (e.g., “sprinkler” mentioned)
High	Very confident	Multiple indicators present but not explicit
Needs Review	Uncertain	Ambiguous or conflicting evidence → flagged for human expert

Real Example Output

Input: 12 documents (Fire Strategy, Structural Report, MEP Spec, etc.)

Output:

```
{
  "deterministicResults": [
    {
      "matrixId": "GW2-FS-001",
      "ruleName": "Fire Strategy Document Presence",
      "category": "Documentation",
      "severity": "high",
      "result": {
        "passed": true,
        "confidence": "definitive",
        "reasoning": "Fire Strategy document identified:
        Fire_Strategy_Report_v2.pdf"
      }
    },
    {
      "matrixId": "GW2-FS-008",
      "ruleName": "Sprinkler System - High Rise",
      "category": "Fire Safety",
      "severity": "high",
      "result": {
        "passed": true,
        "confidence": "definitive",
        "reasoning": "Building is 24.5m tall and sprinkler system documented"
      }
    },
    {
      "matrixId": "GW2-CON-003",
      "ruleName": "Height Consistency Across Documents",
      "category": "Consistency",
      "severity": "medium",
      "result": {
        "passed": false,
        "confidence": "high",
        "reasoning": "Inconsistent heights found: 24.5m (Fire Strategy) vs 25.0m
        (Planning Drawings)",
        "failureMode": "Documents report different building heights - requires
        clarification"
      }
    }
  ],
  "summary": {
    "totalRules": 55,
    "passed": 42,
    "failed": 11,
    "needsReview": 2
  }
}
```

Processing Time

- **All 55 rules:** 2-5 seconds (deterministic = instant, no API calls)
- **Runs entirely in-process** (no external dependencies)

Stage 5: AI Nuanced Analysis (LLM Assessments)

What Happens

For criteria that require **human-like judgment** (not just binary checks), Claude AI performs nuanced assessments.

Examples of nuanced criteria: - Is the Fire Strategy document well-written and thorough? - Are the fire engineering justifications reasonable? - Is the level of detail appropriate for this building type? - Do the proposed materials align with good practice?

The Technology

File: `packages/backend/src/services/matrix-assessment.ts`

```
async function assessCriterion(
  criterion: MatrixRow,
  packDocs: PackDocument[],
  referenceEvidence: CorpusEvidence | null,
  client: Anthropic
): Promise<AssessmentResult> {

  // Build the LLM prompt
  const prompt = `
You are assessing Gateway 2 submissions for the Building Safety Regulator (BSR).

CRITERION: ${criterion.matrix_id} - ${criterion.matrix_title}
CATEGORY: ${criterion.category}
SEVERITY: ${criterion.severity}

SUCCESS DEFINITION:
${criterion.success_definition}

REGULATORY REFERENCE:
${referenceEvidence?.quote || 'No specific reference provided'}

SUBMISSION DOCUMENTS:
${packDocs.map(d => `${d.filename}: ${d.extractedText.slice(0,
        5000)}`).join('\n\n')}

Assess whether this submission meets the criterion.

Return JSON:
{
  "status": "meets" | "partial" | "does_not_meet",
  "reasoning": "Detailed explanation of your assessment",
  "pack_evidence": {
    "found": boolean,
    "document": "filename",
    "quote": "specific quote from submission"
  },
  "gaps_identified": ["gap 1", "gap 2"],
  "proposed_change": "Specific text to insert into document to address gaps (or null
        if requires human intervention)",
  "confidence": "high" | "medium" | "low"
}
  `;

  // Call Claude API
  const response = await client.messages.create({
    model: 'claude-sonnet-4-20250514',
    max_tokens: 4096,
    messages: [{
```

```
      role: 'user',
      content: prompt
    }]
  });

  const result = JSON.parse(response.content[0].text);

  return {
    matrix_id: criterion.matrix_id,
    matrix_title: criterion.matrix_title,
    category: criterion.category,
    status: result.status,
    severity: criterion.severity,
    reasoning: result.reasoning,
    pack_evidence: result.pack_evidence,
    reference_evidence: referenceEvidence,
    gaps_identified: result.gaps_identified,
    actions_required: result.gaps_identified.map(gap => ({
      action: gap,
      owner: 'Project Team',
      effort: criterion.severity === 'high' ? 'L' : 'M',
      expected_benefit: `Address ${criterion.category} compliance gap`
    })),
    confidence: result.confidence,
    proposed_change: result.proposed_change
  };
}
```

Corpus Retrieval (Reference Standards)

The system searches a **regulatory corpus** (stored reference documents: BS 9991, Approved Document B, etc.) to provide context to the AI.

```
async function getCorpusEvidence(criterion: MatrixRow): Promise<CorpusEvidence |
        null> {
  // Search regulatory corpus for relevant passages
  const results = await searchCorpus(
    criterion.reference_sources,  // e.g., ["BS 9991", "Approved Document B"]
    criterion.matrix_title,        // e.g., "Evacuation Strategy"
    limit: 3
  );

  if (results.length === 0) return null;

  return {
    doc_id: results[0].doc_id,
    doc_title: results[0].doc_title,
    page: results[0].page,
    quote: results[0].quote
  };
}
```

What AI Does vs Doesn't

✅ **AI DOES:** - Assess whether information is sufficiently detailed - Evaluate reasonableness of engineering justifications - Identify gaps in documentation - **Generate proposed text changes** to fix issues

❌ **AI DOES NOT:** - Override deterministic rule failures - Make final compliance decisions (human approval required) - Generate text for issues requiring new documents/testing

Proposed Change Generation

Key Feature: The AI can generate **specific, insertable text** to fix gaps.

Example:

Criterion: Fire alarm system specifications insufficient

AI-Generated Proposed Change:

```
The fire alarm system will be designed and installed in accordance with BS 5839-
1:2017
for Category L1 coverage (detection in all areas). The system will include:

• Manual call points at all exit points and within 45m horizontal travel distance
• Automatic smoke detectors in all circulation spaces, common areas, and riser
cupboards
• Heat detectors in plant rooms and service areas
• Audible sounders providing a minimum of 65dB(A) or 5dB(A) above ambient noise
levels
• Visual alarm devices (VADs) in common areas for accessibility compliance
• A two-stage alarm sequence with investigation time in accordance with BS 5839-1
Clause 25.2
• Connection to an Alarm Receiving Centre (ARC) for 24/7 monitoring

The fire alarm panel will be located in the main entrance lobby with remote
indication
panels at secondary access points. All components will be connected via a fault-
tolerant
loop configuration to ensure system resilience.

Regular testing and maintenance will be undertaken by a qualified fire alarm
engineer in
accordance with BS 5839-1 Clause 45, with weekly panel checks and quarterly full
system tests.
```

How it's used: - User reviews this proposed change in the carousel UI - If approved → automatically inserted into document - If skipped → listed in "Outstanding Issues" report

Human Intervention Detection

Some issues **cannot** be fixed by text insertion. The system detects these:

```
const HUMAN_INTERVENTION_KEYWORDS = [
  'new document',
  'new report',
  'create a',
  'commission',
  'specialist',
  'expert',
  'testing',
  'certification',
  'physical',
  'inspection'
];

function requiresHumanIntervention(proposed_change: string): boolean {
  const lower = proposed_change.toLowerCase();
  return HUMAN_INTERVENTION_KEYWORDS.some(kw => lower.includes(kw));
}
```

Example requiring human intervention:

```
{
  "gaps_identified": ["No BS 8414 test certificate provided for external wall
        system"],
  "proposed_change": null,
  "reasoning": "This gap requires obtaining physical test certification from an
        accredited laboratory. Cannot be resolved through text amendments alone."
}
```

Real Example

Input: Criterion GW2-FS-015: “Evacuation Strategy Appropriateness”

AI Assessment:

```
{
  "matrix_id": "GW2-FS-015",
  "status": "partial",
  "reasoning": "The submission states a 'stay put' strategy, which is appropriate
        for this building type. However, the justification lacks detail on why
        simultaneous evacuation was rejected, and there is no discussion of the
        evacuation strategy for basement levels.",
  "pack_evidence": {
    "found": true,
    "document": "Fire_Strategy_Report.pdf",
    "quote": "The building will adopt a stay put evacuation strategy in accordance
        with BS 9991:2015"
  },
  "gaps_identified": [
    "No justification for rejection of simultaneous evacuation",
    "Basement evacuation strategy not addressed"
  ],
  "proposed_change": "The stay put strategy was selected over simultaneous
        evacuation due to the provision of 60-minute fire-resistant compartmentation
        between flats, automatic fire detection and alarm systems, and protected
        means of escape compliant with BS 9991. Simultaneous evacuation would result
        in congestion in staircases and is not necessary given the robust passive
        and active fire protection measures.\n\nFor basement levels, a separate
        evacuation strategy has been developed. Basement occupants will follow a
        phased evacuation procedure, with the basement level evacuating first upon
        alarm activation, followed by ground floor and above if required. This
        approach accounts for the limited vertical travel distance and ensures
        adequate egress capacity.",
  "confidence": "medium"
}
```


Processing Time

- **Per LLM criterion:** 10-20 seconds
- **Total LLM assessments:** 13 criteria
- **Total time:** 130-260 seconds (run in parallel where possible)

Stage 6: Combined Results & Readiness Scoring

What Happens

Results from **Deterministic Rules** (Stage 4) and **AI Analysis** (Stage 5) are combined into a single unified assessment.

The Technology

File: `packages/backend/src/services/matrix-assessment.ts`

```
// Combine both result sets
const allResults = [
  ...deterministicAssessmentResults,  // 55 rules
  ...llmResults                       // 13 LLM assessments
];

// Calculate summary statistics
const assessed = allResults.filter(r => r.status !== 'not_assessed').length;
const meets = allResults.filter(r => r.status === 'meets').length;
const partial = allResults.filter(r => r.status === 'partial').length;
const doesNotMeet = allResults.filter(r => r.status === 'does_not_meet').length;

// Calculate readiness score (weighted)
// Full pass = 1.0, Partial pass = 0.5, Fail = 0.0
const totalCriteria = allResults.length;
const weightedScore = meets + (partial * 0.5);
const readinessScore = Math.round((weightedScore / totalCriteria) * 100);

const assessment: FullAssessment = {
  summary: {
    assessed,
    not_assessed: allResults.length - assessed,
    meets,
    partial,
    does_not_meet: doesNotMeet,
    flagged_high: allResults.filter(r => r.status !== 'meets' && r.severity ===
        'high').length,
    flagged_medium: allResults.filter(r => r.status !== 'meets' && r.severity ===
        'medium').length,
    readiness_score: readinessScore
  },
  results: allResults,
  guardrails: {
    deterministic_rules_count: 55,
    llm_assessments_count: 13,
    reference_corpus_backed: corpusBackedCount,
    reference_anchors_found: withReferenceAnchor
  }
};
```

Readiness Score Calculation

```
Readiness Score = (Full Passes + Partial Passes × 0.5) / Total Criteria × 100

Example:
- 68 total criteria (55 deterministic + 13 LLM)
- 42 full passes (meets)
- 18 partial passes (partial)
- 8 failures (does_not_meet)

Score = (42 + 18 × 0.5) / 68 × 100
      = (42 + 9) / 68 × 100
      = 51 / 68 × 100
      = 75% Readiness
```

Severity Flagging

```
// High-severity issues
const highSeverityIssues = allResults.filter(r =>
  r.status !== 'meets' && r.severity === 'high'
);

// Medium-severity issues
const mediumSeverityIssues = allResults.filter(r =>
  r.status !== 'meets' && r.severity === 'medium'
);

// Low-severity issues (nice-to-haves)
const lowSeverityIssues = allResults.filter(r =>
  r.status !== 'meets' && r.severity === 'low'
);
```

Data Storage

```
-- Store full assessment as JSON in database
UPDATE packVersions
SET matrixAssessment = '{
  "summary": { ... },
  "results": [ ... ],
  "guardrails": { ... }
}'
WHERE id = 'pack-version-123';
```

Real Example Output


```
{
  "summary": {
    "assessed": 68,
    "not_assessed": 0,
    "meets": 42,
    "partial": 18,
    "does_not_meet": 8,
    "flagged_high": 5,
    "flagged_medium": 16,
    "flagged_low": 5,
    "readiness_score": 75
  },
  "results": [
    {
      "matrix_id": "GW2-FS-001",
      "matrix_title": "Fire Strategy Document Presence",
      "category": "Documentation",
      "status": "meets",
      "severity": "high",
      "source": "deterministic"
    },
    {
      "matrix_id": "GW2-FS-008",
      "matrix_title": "Sprinkler System - High Rise",
      "category": "Fire Safety",
      "status": "meets",
      "severity": "high",
      "source": "deterministic"
    },
    {
      "matrix_id": "GW2-FS-015",
      "matrix_title": "Evacuation Strategy Appropriateness",
      "category": "Fire Safety",
      "status": "partial",
      "severity": "medium",
      "source": "llm",
      "proposed_change": "The stay put strategy was selected over..."
    },
    {
      "matrix_id": "GW2-CON-003",
      "matrix_title": "Height Consistency",
      "category": "Consistency",
      "status": "does_not_meet",
      "severity": "medium",
      "source": "deterministic",
```

```
      "proposed_change": null  // Human intervention required
    }
  ],
  "guardrails": {
    "deterministic_rules_count": 55,
    "llm_assessments_count": 13,
    "reference_corpus_backed": 12,
    "reference_anchors_found": 11
  }
}
```

Stage 7: AI Document Generation (Amended Docs + PDFs)

What Happens

After user reviews and approves changes in the carousel UI, the system generates: 1. **Amended Word Document** (.docx) with changes highlighted 2. **PDF version** of the amended document 3. **Outstanding Issues Report** for skipped criteria

The Technology

File: `packages/backend/src/services/document-amendment.ts`

7.1 User Review in Carousel

Frontend: User sees each issue one-by-one in carousel

```
interface CarouselItem {
  matrix_id: string;
  matrix_title: string;
  status: 'meets' | 'partial' | 'does_not_meet';
  pack_evidence: { quote: string };
  reference_evidence: { quote: string };
  proposed_change: string | null;
  requires_human_intervention: boolean;
}
```

User actions: - **Accept** → Add to `acceptedChanges[]` - **Skip** → Add to `skippedCriteriaIds[]`

7.2 Generate Amended Word Document

```
import { Document, Packer, Paragraph, TextRun, AlignmentType, HeadingLevel } from
        'docx';

async function generateAmendedDocx(
  packVersion: PackVersion,
  assessment: FullAssessment,
  acceptedChanges: AcceptedChange[],
  skippedCriteriaIds: string[]
): Promise<string> {

  // Build document sections
  const sections = [];

  // Cover page
  sections.push(
    new Paragraph({
      children: [
        new TextRun({
          text: 'BSR Gateway 2 Submission Pack',
          bold: true,
          size: 32,
          color: '1e3a5f'
        })
      ],
      heading: HeadingLevel.HEADING_1,
      alignment: AlignmentType.CENTER
    }),
    new Paragraph({
      children: [
        new TextRun({
          text: `Pack: ${packVersion.pack.name}`,
          size: 24
        })
      ],
      alignment: AlignmentType.CENTER,
      spacing: { before: 200 }
    }),
    new Paragraph({
      children: [
        new TextRun({
          text: `Amended Version - ${new Date().toLocaleDateString()}`,
          size: 20,
          italics: true
        })
      ],
```

```
    alignment: AlignmentType.CENTER,
    spacing: { before: 100 }
  })
);

// Table of contents
sections.push(
  new Paragraph({
    children: [new TextRun({ text: 'Table of Contents', bold: true, size: 28 })],
    heading: HeadingLevel.HEADING_1,
    spacing: { before: 600, after: 300 }
  })
);

// Group accepted changes by document
const changesByDocument = groupChangesByDocument(acceptedChanges, assessment);

for (const [documentName, changes] of Object.entries(changesByDocument)) {
  sections.push(
    new Paragraph({
      children: [
        new TextRun({
          text: documentName,
          bold: true,
          size: 24,
          color: '1e3a5f'
        })
      ],
      heading: HeadingLevel.HEADING_2,
      spacing: { before: 400, after: 200 }
    })
  );

  for (const change of changes) {
    // Section header
    sections.push(
      new Paragraph({
        children: [
          new TextRun({
            text: `${change.matrix_id}: ${change.matrix_title}`,
            bold: true,
            size: 22
          })
        ],
        heading: HeadingLevel.HEADING_3,
        spacing: { before: 300, after: 100 }
      })
    );
```

```
// Original text (if available)
if (change.original_quote) {
  sections.push(
    new Paragraph({
      children: [
        new TextRun({
          text: 'Original text:',
          bold: true,
          size: 20
        })
      ],
      spacing: { before: 100 }
    }),
    new Paragraph({
      children: [
        new TextRun({
          text: change.original_quote,
          italics: true,
          size: 20
        })
      ],
      spacing: { before: 50, after: 100 }
    })
  );
}

// Amended text (highlighted in yellow)
sections.push(
  new Paragraph({
    children: [
      new TextRun({
        text: 'Amended text:',
        bold: true,
        size: 20
      })
    ],
    spacing: { before: 100 }
  }),
  new Paragraph({
    children: [
      new TextRun({
        text: change.proposed_change,
        size: 20,
        highlight: 'yellow'  // Yellow highlighting for changes
      })
    ],
    spacing: { before: 50, after: 200 },
```

```
          shading: {
            type: ShadingType.CLEAR,
            fill: 'FEF08A'  // Light yellow background
          }
        })
      );

      // Regulatory reference
      if (change.reference_quote) {
        sections.push(
          new Paragraph({
            children: [
              new TextRun({
                text: 'Regulatory reference:',
                bold: true,
                size: 18,
                color: '64748b'
              })
            ],
            spacing: { before: 100 }
          }),
          new Paragraph({
            children: [
              new TextRun({
                text: change.reference_quote,
                size: 18,
                color: '64748b',
                italics: true
              })
            ],
            spacing: { before: 50, after: 300 }
          })
        );
      }
    }
  }

  // Create Word document
  const doc = new Document({
    sections: [{
      properties: {
        page: {
          margin: {
            top: convertInchesToTwip(1),
            right: convertInchesToTwip(1),
            bottom: convertInchesToTwip(1),
            left: convertInchesToTwip(1)
          }
```

```
        }
      },
      headers: {
        default: new Header({
          children: [
            new Paragraph({
              children: [
                new TextRun({
                  text: `${packVersion.pack.name} - Amended Submission`,
                  size: 18,
                  color: '64748b'
                })
              ],
              alignment: AlignmentType.CENTER
            })
          ]
        })
      },
      footers: {
        default: new Footer({
          children: [
            new Paragraph({
              children: [
                new TextRun({ text: 'Page ', size: 18 }),
                new TextRun({
                  children: [PageNumber.CURRENT],
                  size: 18
                }),
                new TextRun({ text: ' of ', size: 18 }),
                new TextRun({
                  children: [PageNumber.TOTAL_PAGES],
                  size: 18
                })
              ],
              alignment: AlignmentType.CENTER
            })
          ]
        })
      },
      children: sections
    }]
  });

  // Write to file
  const buffer = await Packer.toBuffer(doc);
  const filename = `${packVersion.pack.name.replace(/[^a-zA-Z0-9]/g, '-
        ')}_Amended_v${packVersion.versionNumber}.docx`;
  const filepath = path.join(REPORTS_DIR, filename);
```

```
  fs.writeFileSync(filepath, buffer);

  return filepath;
}
```

7.3 Generate PDF Version

```
import puppeteer from 'puppeteer';
import { marked } from 'marked';

async function generateAmendedPdf(
  packVersion: PackVersion,
  assessment: FullAssessment,
  acceptedChanges: AcceptedChange[]
): Promise<string> {

  // Convert changes to markdown
  let markdown = `# BSR Gateway 2 Submission Pack\n\n`;
  markdown += `**Pack:** ${packVersion.pack.name}\n`;
  markdown += `**Amended Version:** ${new Date().toLocaleDateString()}\n\n`;
  markdown += `---\n\n`;
  markdown += `## Table of Contents\n\n`;

  const changesByDocument = groupChangesByDocument(acceptedChanges, assessment);

  for (const [documentName, changes] of Object.entries(changesByDocument)) {
    markdown += `\n## ${documentName}\n\n`;

    for (const change of changes) {
      markdown += `### ${change.matrix_id}: ${change.matrix_title}\n\n`;

      if (change.original_quote) {
        markdown += `**Original text:**\n\n`;
        markdown += `> ${change.original_quote}\n\n`;
      }

      markdown += `**Amended text:**\n\n`;
      markdown += `<div style="background-color: #FEF08A; padding: 15px; border-
        radius: 5px;">\n\n`;
      markdown += `${change.proposed_change}\n\n`;
      markdown += `</div>\n\n`;

      if (change.reference_quote) {
        markdown += `**Regulatory reference:** *${change.reference_quote}*\n\n`;
      }

      markdown += `---\n\n`;
    }
  }

  // Convert markdown to HTML
  const html = `
```

```
    <!DOCTYPE html>
    <html>
    <head>
      <meta charset="UTF-8">
      <style>
        body {
          font-family: 'Helvetica Neue', Arial, sans-serif;
          line-height: 1.6;
          color: #1a1a2e;
          max-width: 800px;
          margin: 40px auto;
          padding: 20px;
        }
        h1 {
          color: #1e3a5f;
          border-bottom: 3px solid #3b82f6;
          padding-bottom: 10px;
        }
        h2 {
          color: #1e3a5f;
          border-bottom: 1px solid #e5e7eb;
          padding-bottom: 5px;
          margin-top: 30px;
        }
        h3 {
          color: #3b82f6;
          margin-top: 25px;
        }
        blockquote {
          border-left: 4px solid #cbd5e1;
          padding-left: 15px;
          color: #64748b;
          font-style: italic;
        }
        code {
          background-color: #f3f4f6;
          padding: 2px 6px;
          border-radius: 3px;
        }
      </style>
    </head>
    <body>
      ${marked(markdown)}
    </body>
    </html>
  `;

  // Convert HTML to PDF using Puppeteer
```

```
  const browser = await puppeteer.launch({
    headless: true,
    args: ['--no-sandbox', '--disable-setuid-sandbox']
  });

  const page = await browser.newPage();
  await page.setContent(html, { waitUntil: 'networkidle0' });

  const filename = `${packVersion.pack.name.replace(/[^a-zA-Z0-9]/g, '-
        ')}_Amended_v${packVersion.versionNumber}.pdf`;
  const filepath = path.join(REPORTS_DIR, filename);

  await page.pdf({
    path: filepath,
    format: 'A4',
    margin: { top: '20mm', right: '15mm', bottom: '20mm', left: '15mm' },
    printBackground: true,
    displayHeaderFooter: true,
    headerTemplate: `
      <div style="font-size: 9px; width: 100%; text-align: center; color: #666;">
        ${packVersion.pack.name} - Amended Submission
      </div>
    `,
    footerTemplate: `
      <div style="font-size: 9px; width: 100%; text-align: center; color: #666;">
        Page <span class="pageNumber"></span> of <span class="totalPages"></span>
      </div>
    `
  });

  await browser.close();

  return filepath;
}
```

7.4 Generate Outstanding Issues Report

```
async function generateOutstandingIssuesReport(
  packVersion: PackVersion,
  assessment: FullAssessment,
  skippedCriteriaIds: string[]
): Promise<string> {

  // Filter to skipped criteria
  const skippedCriteria = assessment.results.filter(r =>
    skippedCriteriaIds.includes(r.matrix_id)
  );

  // Build markdown report
  let markdown = `# Outstanding Issues Report\n\n`;
  markdown += `**Pack:** ${packVersion.pack.name}\n`;
  markdown += `**Date:** ${new Date().toLocaleDateString()}\n\n`;
  markdown += `---\n\n`;
  markdown += `## Summary\n\n`;
  markdown += `Total outstanding issues: **${skippedCriteria.length}**\n\n`;

  const highSeverity = skippedCriteria.filter(c => c.severity === 'high');
  const mediumSeverity = skippedCriteria.filter(c => c.severity === 'medium');
  const lowSeverity = skippedCriteria.filter(c => c.severity === 'low');

  markdown += `- **High severity:** ${highSeverity.length}\n`;
  markdown += `- **Medium severity:** ${mediumSeverity.length}\n`;
  markdown += `- **Low severity:** ${lowSeverity.length}\n\n`;
  markdown += `---\n\n`;

  // High severity issues first
  if (highSeverity.length > 0) {
    markdown += `## High Severity Issues\n\n`;
    for (const issue of highSeverity) {
      markdown += generateIssueSection(issue);
    }
  }

  // Medium severity
  if (mediumSeverity.length > 0) {
    markdown += `## Medium Severity Issues\n\n`;
    for (const issue of mediumSeverity) {
      markdown += generateIssueSection(issue);
    }
  }

  // Low severity
```

```
  if (lowSeverity.length > 0) {
    markdown += `## Low Severity Issues\n\n`;
    for (const issue of lowSeverity) {
      markdown += generateIssueSection(issue);
    }
  }

  // Write to file
  const filename = `${packVersion.pack.name.replace(/[^a-zA-Z0-9]/g, '-
        ')}_Outstanding_Issues_v${packVersion.versionNumber}.md`;
  const filepath = path.join(REPORTS_DIR, filename);

  fs.writeFileSync(filepath, markdown);

  return filepath;
}

function generateIssueSection(issue: AssessmentResult): string {
  let section = `### ${issue.matrix_id}: ${issue.matrix_title}\n\n`;
  section += `**Category:** ${issue.category}\n`;
  section += `**Status:** ${issue.status}\n\n`;
  section += `**Assessment:**\n${issue.reasoning}\n\n`;

  if (issue.gaps_identified.length > 0) {
    section += `**Gaps Identified:**\n`;
    for (const gap of issue.gaps_identified) {
      section += `- ${gap}\n`;
    }
    section += `\n`;
  }

  if (issue.actions_required.length > 0) {
    section += `**Actions Required:**\n`;
    for (const action of issue.actions_required) {
      section += `- **Action:** ${action.action}\n`;
      section += `  - **Owner:** ${action.owner}\n`;
      section += `  - **Effort:** ${action.effort}\n`;
      section += `  - **Expected Benefit:** ${action.expected_benefit}\n`;
    }
    section += `\n`;
  }

  if (issue.proposed_change) {
    section += `**Why Skipped:**\n`;
    section += `This issue was flagged for human intervention because it
        requires:\n`;
    section += `- New documentation or reports\n`;
    section += `- Physical testing or certification\n`;
```

```
    section += `- Specialist input or expert review\n\n`;
  }

  section += `---\n\n`;

  return section;
}
```

Real Example Output

User accepts 18 changes, skips 8:

Generated files:

1. **Riverside_Tower_Amended_v1.docx** (245 KB)
   - Cover page
   - Table of contents
   - 18 sections with original + amended text
   - Yellow highlighting on all changes
   - Regulatory references included
   - Headers/footers with page numbers
2. **Riverside_Tower_Amended_v1.pdf** (312 KB)
   - Same content as DOCX
   - Professional PDF formatting
   - Printer-ready for submission
3. **Riverside_Tower_Outstanding_Issues_v1.md** (18 KB)
   - 8 skipped issues listed
   - Grouped by severity (5 high, 3 medium)
   - Action items with owners
   - Effort estimates for each issue

Processing Time

- **Word document generation:** 5-10 seconds
- **PDF generation:** 10-15 seconds (Puppeteer rendering)
- **Outstanding issues report:** 1-2 seconds

- **Total:** 16-27 seconds (run in parallel)

Complete End-to-End Summary

Full Timeline

```
User uploads 12 PDFs (10 MB total)
         ↓ 10-20 seconds
Stage 1: Text extraction complete
         ↓ 5-10 seconds
Stage 2: Chunking & classification complete
         ↓ 60-180 seconds (parallel)
Stage 3: AI field extraction complete (12 docs × 5-15 sec each)
         ↓ 2-5 seconds
Stage 4: 55 deterministic rules evaluated
         ↓ 130-260 seconds (parallel)
Stage 5: 13 LLM assessments complete
         ↓ 1-2 seconds
Stage 6: Results combined, readiness score calculated
         ↓ User reviews in carousel UI
Stage 7: Amended documents generated (16-27 seconds)
         ↓
Download ready: DOCX + PDF + Outstanding Issues Report

TOTAL TIME: 3-5 minutes
```

Data Flow

```
PDFs (disk)
  → Text (memory)
    → Chunks (database)
      → Extracted Fields (database)
        → Rule Results (memory)
          → LLM Assessments (memory)
            → Combined Assessment (database as JSON)
              → User Review (frontend)
                → Amended Documents (disk)
```

Technology Stack Summary

Stage	Technology	Purpose
PDF Parsing	`pdf-parse`	Extract text from PDFs
Text Chunking	Custom TypeScript	Split text for processing
AI Extraction	Claude Sonnet 4.5 API	Extract structured data
Deterministic Rules	Pure TypeScript (3,627 lines)	Binary pass/fail checks
LLM Analysis	Claude Sonnet 4.5 API	Nuanced assessments
Data Storage	PostgreSQL + Prisma ORM	Persist all data
Document Generation	`docx` library	Create Word docs
PDF Rendering	Puppeteer + Chrome	HTML → PDF conversion
API Framework	Express.js + TypeScript	Backend server
Frontend	React + Vite + Tailwind	User interface

Cost Breakdown (Per Pack)

Operation	Cost	Notes
AI Field Extraction (12 docs)	$36	12 × $3 per doc
Deterministic Rules	$0	Pure logic, no API calls
LLM Assessments (13 criteria)	$26	13 × $2 per assessment
Document Generation	$0	Local processing
Total per pack	**~$62**	At enterprise pricing

Reliability Guarantees

1. **Deterministic Rules:** 100% reproducible (same input = same output)
2. **AI Extraction:** Confidence-scored, low-confidence flagged for review
3. **LLM Assessments:** Reference corpus-backed, human override available
4. **Audit Trail:** Every decision fully traceable to source

5. **Data Integrity:** All data persisted to database, no data loss

---

Appendix: Key Code Files

File	Purpose	Lines of Code
`ingestion.ts`	PDF parsing, chunking, classification	423
`analysis.ts`	Orchestrates extraction & assessment	389
`deterministic-rules.ts`	55 regulatory rules	3,627
`matrix-assessment.ts`	LLM assessment engine	988
`document-amendment.ts`	Word/PDF generation	1,247
`claude.ts`	Claude API wrapper	60
`extractFields.ts`	AI extraction prompts	106
`generateReport.ts`	LLM assessment prompts	164

Total Backend Code: ~7,000 lines of TypeScript

Document Version: 1.0 **Last Updated:** 2026-02-27 **Author:** Attlee Technical Team
Classification: Technical Documentation